

```
$ onboard mksglu/context-mode --profile dev
```

context-mode : un serveur MCP et des hooks qui gardent la sortie brute des outils hors du contexte de l'agent, actif et bien testé mais porté par un seul mainteneur : contribue petit, avec tests

Context window optimization for AI coding agents. Sandboxes tool output (98% reduction), persists session memory, and enforces routing across 17 platforms via MCP + hooks.

21 624

ÉTOILES GITHUB

10

CONTRIBUTEURS ACTIFS
sur 12 semaines

107

PR OUVERTES

8 sept. 2026

DERNIER COMMIT

Que fait ce projet et pour qui ?

> Un serveur MCP plus des hooks pour agents de code, 21624 étoiles et 1555 forks depuis le 2026-02-23, sous licence ELv2 non OSI : lis `LICENSE` avant tout usage en service managé.

context-mode est un serveur MCP qui met la sortie brute des outils en sandbox hors de la fenêtre de contexte (réduction annoncée de 98 %), persiste une mémoire de session et impose le routage sur 17 plateformes via MCP + hooks.

Quatre axes dans le README : sandbox des outils, continuité de session stockée en SQLite, indexée FTS5 et cherchée en BM25, « think in code » (l'agent écrit un script qui `console.log` seulement le résultat au lieu de lire les fichiers un par un), et aucune contrainte de style sur la réponse finale.

Public : les développeurs qui travaillent avec un agent de code ; les topics couvrent claude-code, codex-cli, copilot, cursor-plugin, kiro, opencode, openclaw, pi-agent, zed-extension. Le README détaille l'installation par plateforme : plugin marketplace pour Claude Code, `~/.gemini/settings.json` pour Gemini CLI, `.vscode/mcp.json` + `.github/hooks/context-mode.json` pour VS Code Copilot.

Chiffres : 21624 étoiles, 1555 forks, 226 issues ouvertes, dernier push le 2026-09-08.

Dépôt créé le 2026-02-23 sur un compte utilisateur (pas une organisation), non archivé, branche par défaut `main`.

Licence Elastic License 2.0 (ELv2), non OSI : lire `LICENSE` avant tout usage en service managé ou redistribution.

Comment le code est-il organisé ?

> Tout le TypeScript est dans `src/` (serveur, CLI, adaptateurs), les hooks sont du JS pur dans `hooks/` sans build, et `start.mjs` charge `server.bundle.mjs` avant `build/` : `rm server.bundle.mjs` en dev local, sinon tes changements ne sont jamais chargés.

- └ `.agents/` config agents IA
- └ `.claude-plugin/` manifeste plugin Claude Code
- └ `.claude/` config locale Claude Code
- └ `.codex-plugin/` manifeste plugin Codex
- └ `.cursor-plugin/` manifeste plugin Cursor
- └ `.github/` 5 workflows GitHub Actions
- └ `.openclaw-plugin/` manifeste plugin OpenClaw
- └ `.pi/` extension pour Pi
- └ `bin/` script `statusline.mjs`
- └ `configs/` configs par plateforme
- └ `docs/` documentation
- └ `hooks/` hooks par plateforme, bundles session
- └ `scripts/` scripts build, install, version
- └ `skills/` skills pour agents
- └ `src/` TypeScript : serveur MCP, CLI, adaptateurs
- └ `tests/` tests vitest

... et 20 autres entrées à la racine du dépôt

`src/` : TypeScript du serveur MCP, de la CLI et des adaptateurs ; `tsc` compile vers `build/`, puis esbuild bundle `src/server.ts` en `server.bundle.mjs` et `src/cli.ts` en `cli.bundle.mjs` à la racine.

Fichiers clés de `src/` : `server.ts` (serveur MCP, handlers d'outils), `store.ts` (FTS5), `executor.ts` (exécuteur polyglotte), `security.ts` (règles deny/allow), `session/{db,extract,snapshot}.ts`, `adapters/<plateforme>/` (claude-code, gemini-cli, opencode, codex, vscode-copilot, omp, qwen-code).

`hooks/` : hooks en JS pur (`.mjs`), aucun build : `pretooluse.mjs` (routage), `posttooluse.mjs` (capture d'événements), `precompact.mjs` (snapshot de reprise), `sessionstart.mjs`, `stop.mjs`, un sous-dossier par plateforme et des bundles `session-*.bundle.mjs` produits par esbuild.

`configs/` : fichiers d'installation par plateforme, servis tels quels ; `tests/` : vitest ; `scripts/` : build, install, version ; `docs/` : adr/, `adapters/`, `platform-support.md`.

Un manifeste de plugin par client à la racine : `.claude-plugin/`, `.codex-plugin/`, `.cursor-plugin/`, `.openclaw-plugin/`, `.pi/`, `openclaw.plugin.json` ; `.github/` porte 5 workflows.

`start.mjs` charge `server.bundle.mjs` s'il existe, sinon `build/server.js` : en dev local, `rm server.bundle.mjs`, sinon tes modifications de `src/` ne sont jamais chargées.

Par où commencer à lire ?

```
> Ouvre src/server.ts (handlers des outils ctx_* ), puis src/adapters/opencode/index.ts comme modèle d'un HookAdapter , puis hooks/sessionstart.mjs
→ src/session/db.ts pour la continuité de session.
```

src/server.ts : le serveur MCP et ses handlers d'outils ; npm run dev = npx tsx src/server.ts ; bundlé en server.bundle.mjs .

src/cli.ts : la CLI context-mode (bin → cli.bundle.mjs), commandes setup et doctor ; la CI lance npx tsx src/cli.ts doctor .

src/adapters/opencode/index.ts : cible de main dans package.json, classe OpenCodeAdapter qui implémente HookAdapter , le modèle pour comprendre un adaptateur ; l'interface HookAdapter est dans src/adapters/types.ts , la détection de plateforme dans src/adapters/detect.ts .

start.mjs (lanceur publié avec le paquet npm) et src/adapters/openclaw/index.ts (export ./openclaw) sont déduits du nom, fichiers non lus.

Pour la continuité de session, lire dans l'ordre hooks/sessionstart.mjs → src/session/db.ts → src/session/snapshot.ts : SessionDB persistante dans ~/.claude/context-mode/sessions/<hash>.db , ContentStore éphémère dans /tmp/context-mode-<PID>.db , les événements bruts ne sont jamais injectés dans le contexte, le modèle les retrouve via ctx_search .

Pour la liste et la sémantique des outils ctx_* (sandbox : ctx_batch_execute , ctx_execute , ctx_execute_file , ctx_index , ctx_search , ctx_fetch_and_index ; méta : ctx_stats , ctx_doctor , ctx_upgrade , ctx_purge , ctx_insight), lire la section Install → Claude Code du README, puis leurs handlers dans src/server.ts .

Comment builder, tester et lancer ?

```
> npm install && npm run build && npm test avec node >= 22.5.0 : vitest sur 255 fichiers de test, et la CI rejoue la même chaîne sur ubuntu, macos et windows.
```

Prérequis : node >= 22.5.0 (engines), Bun accepté par le CONTRIBUTING ; better-sqlite3 est un module natif externalisé du bundle ; package.json déclare packageManager: pnpm et le repo contient bun.lock , mais CONTRIBUTING et CI utilisent npm.

Installer puis builder, dans cet ordre : npm install puis npm run build (tsc → build/ , puis npm run bundle , assert-bundle , assert-asymmetric-drift).

Tester : npm test (= vitest run , pretest relance npm run build) ; npm run typecheck (tsc --noEmit) ; npm run test:watch ; 255 fichiers de test sous tests/ , framework vitest.

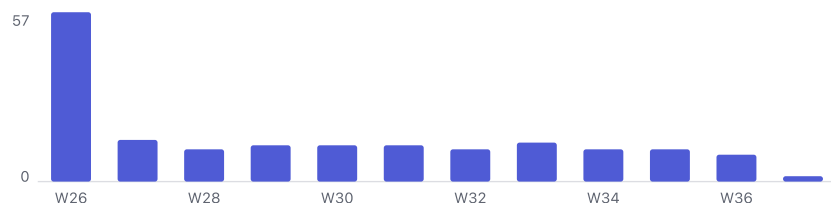
Lancer : en dev npm run dev ; en usage claude mcp add context-mode -- npx -y context-mode (MCP seul, sans hooks) ou /plugin marketplace add mksglu/context-mode puis /plugin install context-mode@context-mode (plugin complet), vérification par /context-mode:ctx-doctor .

Dev local dans Claude Code : symlink de ~/.claude/plugins/cache/context-mode/context-mode/<version> vers ton clone, override du seul hook PreToolUse dans ~/.claude/settings.json (les autres sont déclarés dans hooks/hooks.json), rm server.bundle.mjs , pkill -f "context-mode.*start.mjs" , redémarrer Claude Code ; modifier hooks/*.mjs ou configs/* ne demande pas de rebuild, modifier src/**/*.ts oui.

CI .github/workflows/ci.yml (nom « CI ») : déclenchée par workflow_dispatch, push et pull_request sur main et next , matrice ubuntu/macos/windows, Node 22.5 plus Python, Go et Elixir, étapes npm install → npx tsc -b --noEmit → npm run build → npm run bundle → npm run assert-bundle → vitest → npx tsx src/cli.ts doctor ; les autres workflows (bundle.yml, openclaw-e2e.yml, tier2-e2e-smoke.yml, update-stats.yml) n'ont pas été relevés : à vérifier dans .github/workflows/ .

Comment le projet vit-il ?

> Entre 9 et 14 commits par semaine depuis 2026-W27, mais bus factor 1 : mksglu signe 60 commits, le suivant 6, et 107 PR ouvertes attendent sa revue.



Sur les 12 dernières semaines : pic à 57 commits en 2026-W26, puis entre 9 et 14 par semaine de W27 à W36, et 2 en W37 (semaine en cours) ; dernier commit le 2026-09-08.

Bus factor 1 : mksglu porte 60 commits, ken-jo 6, pg-adm1n 2, les autres contributeurs listés 1 chacun ; le mainteneur se décrit comme « solo maintain with limited time ».

Releases : 10 tags dans les données relevées, de v1.0.160 (2026-06-01) à v1.0.169 (2026-06-29) ; seules les 10 dernières versions ont été relevées, les données ne disent donc pas si une version est sortie depuis juin alors que les commits continuent : à vérifier sur <https://github.com/mksglu/context-mode/releases>.

Flux entrant : 119 issues ouvertes, 6 fermées sur 30 jours, 107 PR ouvertes ; aucune PR fusionnée sur 30 jours n'a été relevée (liste vide), à vérifier sur <https://github.com/mksglu/context-mode/pulls?q=is%3Amerged>.

Les PR ouvertes les plus récentes datent d'aujourd'hui et d'hier (#1138 fix(fetch), #1137 docs, #1129 fix(hooks) ABI) : les contributions externes arrivent, à mettre en regard d'un mainteneur seul pour la revue.

Risques notés : CI low, deps low, licence mid, sécurité mid (pas de SECURITY.md), bus factor high ; pas de CODEOWNERS, un fichier de funding existe.

Quelle première contribution ?

> Pas de good first issue : prends un bug ouvert et discuté (#947 timeout de `ctx_batch_execute` , #1024 nettoyage de DB), écris le test dans le fichier existant du domaine, joins `bash scripts/ctx-debug.sh` , et vérifie d'abord qu'aucune des 107 PR ouvertes ne le couvre.

Aucune issue « good first issue » dans les données relevées (liste vide) ; labels présents : enhancement 6, bug 4, help wanted 1 ; l'entrée sans code est l'issue #45 « Beta testers wanted » (154 commentaires, label help wanted).

Règles du mainteneur : suivre les templates, joindre la sortie de `bash scripts/ctx-debug.sh` et le prompt exact, écrire des tests pour tout changement (TDD red-green-refactor), et ne PAS créer de nouveau fichier de test : compléter celui du domaine (`tests/adapters/<platform>.test.ts` , `tests/core/server.test.ts` , `tests/hooks/*.test.ts` , `tests/session/*.test.ts`).

Bugs ouverts les plus discutés, bons candidats : #947 (le timeout de `ctx_batch_execute` ne borne pas l'indexation, 4 commentaires), #950 (`ctx_stats` incohérent, 5), #1024 (nettoyage de la DB qui supprime des sessions vivantes, 2), #1022 (snapshot de reprise qui ignore le budget d'octets, 2).

Si tu utilises Claude Code : #911 et #946 (6 et 3 commentaires) portent sur le classifieur auto-mode déclenché par le prompt injecté et demandent un opt-out, reproductibles sur ton poste.

Avant d'ouvrir une PR, cherche un doublon parmi les 107 ouvertes (ex. #1138 tables HTML sans lignes, #1127 budgets echo par hôte, #1082 timeout en millisecondes) ; la CI cible `main` et `next` , mais CONTRIBUTING n'a été relevé que jusqu'au début de « Submitting a Pull Request » : la branche de base des PR et la suite du processus sont à vérifier dans CONTRIBUTING.md.

Interdit par la politique « prose-style » : aucune directive de brièveté dans `src/server.ts` , `hooks/routing-block.mjs` ou `configs/*` , un test de régression dans `tests/core/server.test.ts` casse la CI sinon ; si ton changement touche la sortie d'un outil, joins un avant/après dans la PR.

\$ Vos 3 premières actions

1. Clone, bulde, teste : `git clone https://github.com/mksglu/context-mode.git && cd context-mode && npm install && npm run build && npm test` ; il faut node >= 22.5.0, `pretest` relance le build et `vitest` parcourt 255 fichiers de test.
2. Branche ton clone sur Claude Code : `rm server.bundle.mjs` , symlink de `~/.claude/plugins/cache/context-mode/context-mode/<version>` vers le clone, override du seul hook `PreToolUse` dans `~/.claude/settings.json` , version `-dev` dans `package.json` , `src/server.ts` , `.claude-plugin/plugin.json` et `.claude-plugin/marketplace.json` , `npm run build` , `pkill -f "context-mode.*start.mjs"` , redémarre, puis `/context-mode:ctx-doctor` doit afficher ta version `-dev` .
3. Prends #947 (le timeout de `ctx_batch_execute` ne borne pas l'indexation, 4 commentaires) ou #1024 (le nettoyage de la DB supprime des sessions vivantes, 2 commentaires) : test rouge d'abord dans le fichier existant du domaine (`tests/core/server.test.ts` pour les outils, `tests/store.test.ts` pour le store), jamais un nouveau fichier, puis `bash scripts/ctx-debug.sh` et un avant/après de la sortie dans la PR, après avoir vérifié qu'aucune des 107 PR ouvertes ne couvre déjà le sujet.
4. Avant de pousser : `npm run typecheck` puis `npm test` , et aucune directive de brièveté dans `src/server.ts` , `hooks/routing-block.mjs` ou `configs/*` , le test de régression « prose-style policy (#482) » de `tests/core/server.test.ts` casse la CI sinon.